Contents

- 1 Abstract
- 2 Table of Contents
- 3 1. Introduction
  - 3.1 1.1 Motivation
  - 3.2 1.2 Why options markets differ from equity markets
  - 3.3 1.3 Why traditional technical trading is often insufficient for options
  - 3.4 1.4 Long-term vision
- 4 2. Fundamentals of Options
  - 4.1 2.1 What is an option?
  - 4.2 2.2 Call option
  - 4.3 2.3 Put option
  - 4.4 2.4 Intrinsic value and time value
  - 4.5 2.5 Moneyness
  - 4.6 2.6 Expiration
  - 4.7 2.7 Payoff diagrams
  - 4.8 2.8 Simple numerical examples
- 5 3. Option Pricing Theory
  - 5.1 3.1 Arbitrage-free pricing
  - 5.2 3.2 Risk-neutral valuation
  - 5.3 3.3 Stochastic nature of asset prices
  - 5.4 3.4 Volatility as a key input
  - 5.5 3.5 Black-Scholes call and put formulas
  - 5.6 3.6 Intuition behind Black-Scholes
  - 5.7 3.7 Limitations of Black-Scholes
- 6 4. Option Greeks
  - 6.1 4.1 Why Greeks matter
  - 6.2 4.2 Delta
  - 6.3 4.3 Gamma
  - 6.4 4.4 Theta
  - 6.5 4.5 Vega
  - 6.6 4.6 Delta hedging and market makers
  - 6.7 4.7 How the repository implements Greeks
- 7 5. Options Market Microstructure
  - 7.1 5.1 Option chain structure
  - 7.2 5.2 Open interest and change in open interest

- 7.3 5.3 Volume
- 7.4 5.4 Liquidity concentration and strike clustering
- 7.5 5.5 Expiry effects
- 7.6 5.6 Dealers and market makers

- 8 6. Dealer Gamma Exposure
  - 8.1 6.1 Aggregate gamma
  - 8.2 6.2 Long gamma versus short gamma
  - 8.3 6.3 Gamma flip
  - 8.4 6.4 Trading intuition
  - 8.5 6.5 Repository implementation
- 9 7. Dealer Hedging Flows
  - 9.1 7.1 Delta hedging mechanics
  - 9.2 7.2 Feedback loops
  - 9.3 7.3 Pinning behavior
  - 9.4 7.4 Repository implementation
- 10 8. Volatility Dynamics
  - 10.1 8.1 Realized volatility versus implied volatility
  - 10.2 8.2 Volatility surface
  - 10.3 8.3 Compression versus expansion
  - 10.4 8.4 Expected move
  - 10.5 8.5 Why volatility matters for option buying
- 11 9. Liquidity Structure in Options
  - 11.1 9.1 Gamma walls and liquidity walls
  - 11.2 9.2 Liquidity voids and vacuums
  - 11.3 9.3 Clusters and path dependency
- 12 10. Macro and News Analysis
  - 12.1 10.1 Why macro matters for options
  - 12.2 10.2 Scheduled event risk
  - 12.3 10.3 Headline classification
  - 12.4 10.4 Event risk logic
- 13 11. Global Risk Factors
  - 13.1 11.1 Why global markets matter for local index options
  - 13.2 11.2 Cross-asset shock logic
  - 13.3 11.3 Global risk regimes
- 14 12. Volatility Explosion and Convexity
  - 14.1 12.1 Why volatility explosions occur
  - 14.2 12.2 Squeeze dynamics and air pockets

- 14.3 12.3 Gamma-vol feedback
- 15 13. Signal Generation Framework
  - 15.1 13.1 Philosophy
  - 15.2 13.2 Flow signals
  - 15.3 13.3 Gamma regime signals
  - 15.4 13.4 Volatility regime signals
  - 15.5 13.5 Liquidity signals
  - 15.6 13.6 Trade strength scoring
  - 15.7 13.7 Confirmation filters
- 16 14. Trade Construction
  - 16.1 14.1 From signal to trade
  - 16.2 14.2 Direction determination
  - 16.3 14.3 Strike selection
  - 16.4 14.4 Capital constraints
  - 16.5 14.5 Target and stop logic
- 17 15. Research and Backtest Framework
  - 17.1 15.1 Research as signal evaluation
  - 17.2 15.2 Outcome horizons
  - 17.3 15.3 Composite scoring
  - 17.4 15.4 Backtesting
  - 17.5 15.5 Tuning methodology
  - 17.6 15.6 Overfitting danger
- 18 16. System Architecture
  - 18.1 16.1 High-level architecture
  - 18.2 16.2 Analytics module
  - 18.3 16.3 Engine module
  - 18.4 16.4 Strategy module
  - 18.5 16.5 Data module
  - 18.6 16.6 Models module
  - 18.7 16.7 Backtest module
  - 18.8 16.8 Visualization module
  - 18.9 16.9 Risk and overlay modules
  - 18.10 16.10 Tuning and governance modules
- 19 17. Implementation Details
  - 19.1 17.1 Options pricing and Greeks
  - 19.2 17.2 Microstructure state collection
  - 19.3 17.3 Trade strength

- 19.4 17.4 Confirmation layer
- 19.5 17.5 Global risk layer
- 19.6 17.6 Gamma-vol acceleration layer
- 19.7 17.7 Dealer hedging pressure layer
- 19.8 17.8 Option efficiency layer
- 19.9 17.9 Research dataset
- 19.10 17.10 Parameter governance

- 20 18. Limitations
  - 20.1 18.1 Model assumptions
  - 20.2 18.2 Data limitations
  - 20.3 18.3 Parameter sensitivity
  - 20.4 18.4 Execution limitations
  - 20.5 18.5 Regime dependency
  - 20.6 18.6 Structural simplifications
  - 20.7 18.7 Statistical and research limitations
  - 20.8 18.8 Software and operational limitations
- 21 19. Future Enhancements
  - 21.1 19.1 Improved volatility modeling
  - 21.2 19.2 Better dealer positioning estimates
  - 21.3 19.3 Machine learning extensions
  - 21.4 19.4 Improved news analysis
  - 21.5 19.5 Improved option efficiency models
  - 21.6 19.6 Capital allocation systems
  - 21.7 19.7 Portfolio-level and cross-signal research
  - 21.8 19.8 Execution and transaction-cost research
  - 21.9 19.9 Research infrastructure enhancements
- 22 20. Assumptions Used in Interpreting the Repository
- 23 Concluding Remark

1 Abstract

This document presents a thesis-style technical explanation of `options_quant_engine`, a research and live signal-generation system for Indian index and stock options. The engine is built around the idea that option markets are not merely directional forecasting problems. They are convexity markets shaped by volatility, dealer hedging flows, strike concentration, liquidity discontinuities, and macro regime shifts. Traditional technical trading frameworks tend to ignore

these features or absorb them only indirectly. This repository instead treats them as first-class state variables.

The document begins from first principles: the definition of an option, the mathematics of arbitrage-free pricing, the Greeks, dealer hedging behavior, and the microstructure of listed options. It then develops the intuition behind gamma exposure, gamma flips, pinning, liquidity vacuums, volatility compression-expansion transitions, and macro/global risk overlays. These ideas are connected explicitly to the repository architecture and implementation.

The system is also unusual in one important respect: it is signal-evaluation-first. It does not learn from discretionary trade execution. Instead, it generates signals, stores them in a canonical research dataset, and judges them against subsequent market behavior. Parameter tuning, walk-forward validation, promotion, and shadow-mode governance all derive from this signal evaluation loop. This design separates market intelligence from execution noise, avoids contamination from discretionary trading behavior, and creates a clean research-to-production calibration process.

2 Table of Contents

1. Introduction
2. Fundamentals of Options
3. Option Pricing Theory
4. Option Greeks
5. Options Market Microstructure
6. Dealer Gamma Exposure
7. Dealer Hedging Flows
8. Volatility Dynamics
9. Liquidity Structure in Options
10. Macro and News Analysis
11. Global Risk Factors
12. Volatility Explosion and Convexity
13. Signal Generation Framework
14. Trade Construction
15. Research and Backtest Framework
16. System Architecture
17. Implementation Details
18. Limitations
19. Future Enhancements
20. Assumptions Used in Interpreting the Repository

3 1. Introduction

3.1 1.1 Motivation

An options trading engine exists for a different reason than a conventional equity signal engine. In a linear instrument such as a stock or future, the relationship between price movement and profit and loss is approximately proportional:

$$\Delta PnL \approx q \cdot \Delta S$$

where q is position size and ΔS is the underlying move.

In an option, profit and loss is not linear in the underlying. It depends on both the level and the path of the underlying, the time remaining to expiry, and the market's changing estimate of volatility:

$$V = V(S, K, T, \sigma, r, q)$$

where:

- S is spot price
- K is strike
- T is time to expiry
- σ is volatility
- r is the risk-free rate
- q here can also denote dividend yield in a pricing context

This means that options trading is not simply "predict up" or "predict down." It is a structured inference problem involving:

- direction of the move
- magnitude of the move
- timing of the move
- volatility regime
- path dependence
- strike efficiency
- liquidity and dealer positioning

The motivation for building `options_quant_engine` is therefore not to create a faster indicator dashboard, but to create a layered market-state engine that can reason about direction, convexity, liquidity, and exogenous risk simultaneously.

3.2 1.2 Why options markets differ from equity markets

Equity markets are primarily inventory and information markets. Option markets are additionally transfer mechanisms for convexity and volatility. A trade in an option changes not just exposure to price direction, but the sensitivity of that exposure itself.

An option market therefore behaves differently because:

- option prices embed implied volatility as a tradable input
- dealer hedging creates feedback between option positioning and spot movement
- strike-level open interest can create support, resistance, or pinning effects
- near-expiry options can exhibit discontinuous behavior in gamma and theta
- macro events can reprice both direction and volatility at once

This is why a strategy that works on a cash equity chart often performs poorly when mapped naively onto option buying.

3.3 1.3 Why traditional technical trading is often insufficient for options

Purely technical frameworks often assume that:

- price structure contains most relevant information
- volatility is secondary
- execution friction is modest
- the instrument behaves linearly

For options, these assumptions break down. A technically correct directional call can still lose money because:

- implied volatility collapses
- the selected strike is too far out-of-the-money
- time decay overwhelms the move
- the target is unrealistic relative to the expected move
- dealers are long gamma and suppress the very move the chart appears to imply

The system therefore incorporates:

- microstructure analytics
- dealer positioning and hedging logic
- liquidity structure
- macro/news overlays
- global risk regimes
- option efficiency overlays

3.4 1.4 Long-term vision

The long-term vision of the repository is to become a disciplined research-to-production framework for options signal generation and evaluation. It aims to detect:

- directional opportunities
- volatility opportunities
- liquidity dislocations
- dealer-driven market moves

and to do so in a way that is:

- interpretable
- auditable
- tunable
- robust across market regimes
- independent of discretionary execution history

The engine currently lives at the signal layer, not the broker execution layer. This is intentional. The intellectual core of the system is market-state understanding and signal evaluation, not order routing.

4 2. Fundamentals of Options

4.1 2.1 What is an option?

An option is a derivative contract granting the holder the right, but not the obligation, to transact an underlying asset at a pre-specified strike price on or before expiration.

There are two basic types:

- call option: right to buy the underlying at strike K
- put option: right to sell the underlying at strike K

The holder pays a premium up front. The seller receives that premium in exchange for taking on contingent liability.

4.2 2.2 Call option

A call option benefits when the underlying rises above the strike.

At expiry, its intrinsic payoff is:

$$\max(S_T - K, 0)$$

where:

- S_T is the underlying price at expiration
- K is strike

If $S_T \leq K$, the call expires worthless.

4.3 2.3 Put option

A put option benefits when the underlying falls below the strike.

Its intrinsic payoff at expiry is:

$$\max(K - S_T, 0)$$

If $S_T \geq K$, the put expires worthless.

4.4 2.4 Intrinsic value and time value

The premium of an option can be decomposed into:

$$\text{Option Premium} = \text{Intrinsic Value} + \text{Time Value}$$

For a call:

$$\text{Intrinsic Value}_{call} = \max(S - K, 0)$$

For a put:

$$\text{Intrinsic Value}_{put} = \max(K - S, 0)$$

Time value reflects the market's belief that future movement may make the option more valuable before expiration.

4.5 2.5 Moneyness

Moneyness describes the relationship between spot and strike.

For calls:

- in the money (ITM): $S > K$
- at the money (ATM): $S \approx K$
- out of the money (OTM): $S < K$

For puts:

- ITM: $S < K$
- ATM: $S \approx K$
- OTM: $S > K$

Moneyness matters because it changes:

- delta sensitivity
- gamma sensitivity
- premium composition
- convexity
- sensitivity to volatility

4.6 2.6 Expiration

Options have finite life. This is foundational. A correct directional view with incorrect timing can still lose money.

As expiration approaches:

- gamma tends to increase near ATM strikes
- theta decay accelerates
- small spot moves can create large delta changes
- implied volatility and expiry positioning interact more strongly

4.7 2.7 Payoff diagrams

At expiry:

Long call payoff:

$$\Pi_{call}(S_T) = \max(S_T - K, 0) - C_0$$

Long put payoff:

$$\Pi_{put}(S_T) = \max(K - S_T, 0) - P_0$$

where C_0 and P_0 are the premiums paid.

The critical insight is that option payoffs are nonlinear. This nonlinearity is the source of convexity, but also the source of model risk.

4.8 2.8 Simple numerical examples

Examples are useful because the nonlinearity of options often becomes obvious only when one compares them directly to linear instruments.

4.8.1 EXAMPLE 1: LONG CALL

Suppose:

- spot at trade time: $S_0 = 22,000$
- strike: $K = 22,100$
- premium paid: $C_0 = 90$

At expiry:

- if $S_T = 21,950$, payoff is 0, profit is -90
- if $S_T = 22,100$, payoff is 0, profit is -90
- if $S_T = 22,250$, payoff is 150, profit is 60

The important point is that a moderately bullish view is not enough. The move must exceed both the strike hurdle and the premium paid.

4.8.2 EXAMPLE 2: LONG PUT

Suppose:

- spot at trade time: $S_0 = 22,000$
- strike: $K = 21,900$
- premium paid: $P_0 = 80$

At expiry:

- if $S_T = 22,050$, payoff is 0, profit is -80
- if $S_T = 21,900$, payoff is 0, profit is -80
- if $S_T = 21,700$, payoff is 200, profit is 120

Again, the move must be large enough and timely enough to overcome time value paid at entry.

4.8.3 WHY THESE EXAMPLES MATTER FOR THE REPOSITORY

This is the conceptual reason the repository contains:

- strike selection logic in `strategy/strike_selector.py`
- target/stop and trade geometry in `strategy/exit_model.py`
- expected move and option efficiency logic in `risk/option_efficiency_*`

The engine is not satisfied with the statement "market may go up." It also asks whether the chosen option can convert that view into a favorable payoff.

5 3. Option Pricing Theory

5.1 3.1 Arbitrage-free pricing

The theoretical foundation of option pricing is that derivative prices must be consistent with the absence of arbitrage. If a derivative and a replicating portfolio have the same future cash flows, they should have the same current price.

This leads to the principle of replication and risk-neutral valuation.

5.2 3.2 Risk-neutral valuation

Under standard assumptions, the price of an option equals the discounted expected value of its payoff under a risk-neutral probability measure:

$$V_0 = e^{-rT}\mathbb{E}^{\mathbb{Q}}[\text{Payoff}]$$

This does not mean investors are actually risk neutral. It means that in a frictionless arbitrage-free market, prices can be represented as though expected payoffs are taken under a transformed measure.

The intuition is easier to see through replication. If one can continuously rebalance a portfolio of stock and cash so that it matches the derivative's future payoff path-by-path, then the derivative cannot rationally trade at a different price. Otherwise, one could buy the cheaper side and sell the more expensive side for an arbitrage gain.

5.3 3.3 Stochastic nature of asset prices

The Black-Scholes framework assumes that the underlying follows geometric Brownian motion:

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

where:

- μ is drift
- σ is volatility
- W_t is Brownian motion

In risk-neutral form:

$$dS_t = rS_t dt + \sigma S_t dW_t^{\mathbb{Q}}$$

The drift changes from μ to r because under the pricing measure we are not trying to forecast the physical expected return of the asset. We are trying to price contingent claims consistently with no arbitrage.

An important implementation-level intuition follows from this: the pricing model and the signal model are not the same thing. A pricing model can be risk-neutral while the trading engine is explicitly trying to exploit predictive structure in flow, gamma, volatility state, and macro conditions.

5.4 3.4 Volatility as a key input

Volatility is not just noise. In option pricing, volatility determines the distribution of terminal outcomes, and therefore the value of optionality.

Everything else equal, higher volatility raises the value of both calls and puts because the payoff is convex:

$$\mathbb{E}[\max(X, 0)] \geq \max(\mathbb{E}[X], 0)$$

This is a direct consequence of Jensen's inequality.

In practical options trading, this means:

- convex payoffs become more valuable when the distribution widens
- one must care about both realized volatility and implied volatility
- an option can become more expensive even when spot does not move materially

This is why the repository does not treat volatility merely as a backdrop. It has dedicated modules for volatility regime, cross-asset volatility stress, and expected-move efficiency.

5.5 3.5 Black-Scholes call and put formulas

For a call:

$$C = Se^{-qT}N(d_1) - Ke^{-rT}N(d_2)$$

For a put:

$$P = Ke^{-rT}N(-d_2) - Se^{-qT}N(-d_1)$$

where:

$$d_1 = \frac{\ln(S/K) + (r - q + \frac{1}{2}\sigma^2)T}{\sigma \sqrt{T}}$$

$$ d_2 = d_1 - \sigma \sqrt{T} $$

and:

- S = spot price
- K = strike
- T = time to expiry
- r = risk-free rate
- q = dividend yield
- σ = volatility
- $N(\cdot)$ = standard normal CDF

One should also recall put-call parity:

$$C - P = Se^{-qT} - Ke^{-rT}$$

This relation is foundational because it shows that call and put prices are not independent objects. They are linked by spot, strike, time, carry, and financing.

5.5.1 INTUITION FOR D_1 AND D_2

The terms d_1 and d_2 are often taught mechanically, but they carry useful intuition:

- d_1 can be viewed as a risk-adjusted standardized distance to strike
- d_2 is d_1 shifted by one volatility unit over the horizon

Very loosely:

- large positive d_1 means the option is safely in-the-money in standardized terms
- near-zero d_1 means the strike is near the relevant center of the distribution
- large negative d_1 means the option is far out-of-the-money

5.5.2 WHY THIS MATTERS FOR THE ENGINE

The repository does not use Black-Scholes to claim that the world is lognormal. It uses the framework to obtain disciplined local sensitivities such as delta and gamma, which are then fed into microstructure interpretation.

5.6 3.6 Intuition behind Black-Scholes

Black-Scholes can be understood as a continuous-time replication argument. A portfolio of stock and cash can be dynamically adjusted to replicate the option payoff. If replication is possible, the option price must equal the cost of the replicating portfolio.

Its great importance is not that it is perfect, but that it organizes thinking about:

- directional sensitivity
- convexity
- time decay
- volatility sensitivity

Another way to state its importance is this: Black-Scholes is a coordinate system. Even when the actual market violates the assumptions, the model still gives a coherent local language for describing exposure.

5.6.1 A PRACTICAL EXAMPLE

Imagine two option contracts with the same premium but different Greeks:

- contract A: higher delta, lower gamma
- contract B: lower delta, higher gamma

Even if both cost the same, they do not represent the same trade. Contract A is closer to a linear directional instrument. Contract B is more dependent on a fast move and may be more sensitive to volatility repricing. The repository's strike selection and option-efficiency layers exist precisely because such distinctions matter operationally.

5.7 3.7 Limitations of Black-Scholes

Real markets violate key assumptions:

- volatility is not constant
- returns exhibit jumps and skew
- markets are not frictionless
- hedging is discrete, not continuous
- liquidity varies across strikes and time
- large dealer inventories move prices

The repository acknowledges this directly. The Black-Scholes layer in `analytics/greeks_engine.py` is used as a practical sensitivity engine, not as a claim that real markets obey idealized diffusion assumptions.

This implementation choice is important. A live options engine needs

- tractable sensitivities
- consistent cross-strike calculations
- transparent formulas

more than it needs theoretical purity. The repository therefore uses Black-Scholes-style Greeks as a practical state-estimation tool and then adds separate overlays for the parts the model omits:

- liquidity structure
- macro risk
- dealer-flow feedback
- volatility regime changes

6 4. Option Greeks

6.1 4.1 Why Greeks matter

Greeks measure how option value changes with respect to underlying variables. They are the bridge between pricing theory and trading behavior.

6.2 4.2 Delta

Delta is the sensitivity of option price to spot:

$$ \Delta = \frac{\partial V}{\partial S} $$

In Black-Scholes:

For a call:

$$\Delta_{call} = e^{-qT} N(d_1)$$

For a put:

$$\Delta_{put} = e^{-qT} (N(d_1) - 1)$$

Trading significance:

- ATM options have moderate delta
- deep ITM options have delta near 1 in absolute value
- OTM options have low delta but often higher convexity potential

6.2.1 LOCAL APPROXIMATION

For a small underlying move ΔS, option price changes locally as:

$$\Delta V \approx \Delta \cdot \Delta S$$

This approximation is only first order, but it is extremely useful. It tells us whether an option is behaving more like:

- a leveraged directional instrument, or
- a convex lottery on a larger move

6.2.2 TRADING INTUITION

A high-delta option gives more immediate participation in spot movement, but it usually costs more. A low-delta option is cheaper but requires a larger move and usually faster timing. This trade-off is central to the repository's strike selection and option efficiency layers.

6.3 4.3 Gamma

Gamma is the curvature of option value with respect to spot:

$$ \Gamma = \frac{\partial^2 V}{\partial S^2} $$

In Black-Scholes:

$$ \Gamma = \frac{e^{-qT}\phi(d_1)}{S\sigma\sqrt{T}} $$

where $\phi(\cdot)$ is the standard normal PDF.

Gamma matters because it tells us how rapidly delta changes as spot moves.

High gamma means:

- hedges need frequent adjustment
- small spot moves create nonlinear exposure change
- dealer behavior can become mechanically pro-cyclical or counter-cyclical

6.3.1 SECOND-ORDER PRICE APPROXIMATION

For a small move, the option's value can be approximated by:

$$ \Delta V \approx \Delta \cdot \Delta S + \frac{1}{2}\Gamma (\Delta S)^2 $$

This second-order term is why gamma is associated with convexity. The sign of Γ for a long option is positive, meaning large absolute moves become progressively more valuable.

6.3.2 WHY DEALERS CARE ABOUT GAMMA MORE THAN MANY RETAIL TRADERS DO

If a desk is running a large options inventory, gamma is not a small correction. It determines how rapidly the hedge must change. If the desk is effectively short gamma, every move forces re-hedging in the same direction as the move, which can amplify realized volatility.

6.4 4.4 Theta

Theta measures time decay:

$$ \Theta = \frac{\partial V}{\partial t} $$

All else equal, a long option loses value as time passes, especially near expiry.

For option buyers, theta is a cost of being wrong on timing.

More precisely, one can think of theta as the rent paid for optionality. Holding convex exposure is valuable, but that value decays as the time window for favorable outcomes shrinks.

This is one reason the repository emphasizes:

- expected move
- target reachability
- overnight risk

A slow drift in the correct direction may still be a poor long-option trade if theta consumption dominates realized gain.

6.5 4.5 Vega

Vega measures sensitivity to implied volatility:

$$ \nu = \frac{\partial V}{\partial \sigma} $$

In Black-Scholes:

$$ \nu = S e^{-qT}\phi(d_1)\sqrt{T} $$

Vega is central for option buying because a trade can win from:

- direction
- volatility expansion
- or both

6.5.1 A USEFUL DECOMPOSITION

For small changes, a heuristic local expansion is:

$$ \Delta V \approx \Delta \cdot \Delta S + \frac{1}{2}\Gamma (\Delta S)^2 + \nu \cdot \Delta \sigma + \Theta \cdot \Delta t $$

This is not a full valuation identity, but it is a powerful mental model. It shows that option PnL comes from several interacting channels:

- spot movement

- curvature
- volatility repricing
- passage of time

That decomposition is one of the conceptual foundations of the repository's overlay architecture.

6.6 4.6 Delta hedging and market makers

Dealers frequently hedge option positions by trading the underlying or futures. A dealer long option inventory may short futures against positive delta. As spot moves, delta changes, requiring further hedging.

This dynamic is what makes gamma regime analysis so important.

6.7 4.7 How the repository implements Greeks

The repository computes contract-level Greek estimates in `analytics/greeks_engine.py` using Black-Scholes-style formulas and then aggregates them across the chain. The engine uses:

- `DELTA`
- `GAMMA`
- `THETA`
- `VEGA`
- `VANNA`
- `CHARM`

for structural inference rather than textbook valuation alone.

This is a deliberate implementation choice. For example:

- `DELTA` helps map option premium targets into approximate underlying move requirements
- `GAMMA` helps estimate convexity and dealer sensitivity
- `VANNA` and `CHARM` provide additional structure for how hedges may evolve as volatility or time changes

The system therefore uses Greeks in a hybrid way: part valuation language, part microstructure state language.

7 5. Options Market Microstructure

7.1 5.1 Option chain structure

An option chain is a matrix of strikes and expiries containing:

- last traded price
- volume
- open interest
- implied volatility
- bid/ask style pricing information when available

The chain is not just a menu of contracts. It is a map of positioning and liquidity distribution.

7.2 5.2 Open interest and change in open interest

Open interest is the number of outstanding contracts. At a strike level, large open interest often signals:

- concentrated positioning
- hedging significance
- potential pinning
- possible structural support or resistance

Change in open interest can indicate fresh positioning or covering.

An important nuance is that open interest alone does not reveal who is long or short the contract. But at a structural level it still matters, because large inventory concentrations imply that someone will care about price behavior near that strike. The repository uses this kind of structural interpretation rather than claiming perfect participant identification.

7.3 5.3 Volume

Volume tells us where activity is occurring today, while open interest tells us where inventory is already accumulated.

The interaction between the two is informative:

- high volume into high OI can reinforce structural zones
- high volume into low OI may signal new positioning
- large directional flow can align with dealer hedging feedback

7.4 5.4 Liquidity concentration and strike clustering

Options markets often concentrate around:

- round-number strikes
- ATM strikes
- weekly expiry pivots
- large previously traded strikes

This clustering creates localized microstructure effects:

- pinning
- barrier-like behavior
- asymmetric response once the cluster is breached

7.5 5.5 Expiry effects

Near expiry:

- gamma can become concentrated
- theta accelerates
- small spot changes strongly affect hedge flows
- strike pinning becomes more likely
- structural breaks can become more violent if pinning fails

7.6 5.6 Dealers and market makers

A dealer generally provides liquidity by taking the opposite side of customer flow, then hedging net risk dynamically. This creates a distinction between:

- customer directional demand
- dealer inventory
- dealer hedge demand

The engine's architecture explicitly models this distinction.

In implementation terms, this is why the repository separates:

- raw chain analytics in `analytics/`
- trade decision logic in `engine/` and `strategy/`
- overlay inference in `risk/`

It is easier to reason about the system when inventory inference, signal generation, and risk modification remain distinct.

8 6. Dealer Gamma Exposure

8.1 6.1 Aggregate gamma

Aggregate gamma exposure is a proxy for the market's net curvature at the index level. In practice, one estimates:

$$\text{GEX} \approx \sum_i \Gamma_i \cdot OI_i \cdot \text{sign}_i$$

with some convention for sign depending on call/put structure and inferred dealer inventory assumptions.

The repository approximates gamma exposure in `analytics/gamma_exposure.py` using either contract gamma when available or a distance-from-ATM proxy when it is not.

8.1.1 WHY APPROXIMATION IS ACCEPTABLE HERE

Institutional-quality dealer gamma measurement ideally requires:

- reliable contract Greeks
- inventory-side assumptions
- contract multipliers
- expiration-aware aggregation

The repository often operates with public or broker-normalized chain data, so a perfect measure is not always feasible. The design choice is therefore to use a robust proxy that preserves regime information even when the full data quality of an options desk is unavailable.

8.2 6.2 Long gamma versus short gamma

If dealers are effectively long gamma, their hedging tends to oppose price movement:

- sell into rallies
- buy into dips

This creates mean reversion and dampening.

If dealers are effectively short gamma, their hedging tends to reinforce price movement:

- buy into rallies

- sell into selloffs

This creates acceleration and instability.

8.3 6.3 Gamma flip

The gamma flip is the spot level at which net gamma changes sign. It is a structural transition point:

$$\text{Gamma Flip} = \{S : \text{Net Gamma}(S) = 0\}$$

Near the flip:

- hedging sensitivity is high
- small moves can change feedback sign
- regime transitions become plausible

The repository estimates this in `analytics/gamma_flip.py` by aggregating signed strike-wise gamma and interpolating the zero crossing.

8.3.1 INTUITION THROUGH A SIMPLE EXAMPLE

Suppose net gamma by strike is:

- negative below 22,000
- near zero around 22,050
- positive above 22,100

Then the market is near a transition region. If spot is around 22,060, a small move lower may push the market into a short-gamma regime, while a small move higher may place it back into a stabilizing long-gamma regime. The engine does not need the exact true institutional flip level to gain value from knowing that such a transition region exists.

8.4 6.4 Trading intuition

Long gamma suggests:

- mean reversion
- compressed realized movement
- less favorable environment for aggressive option buying

Short gamma suggests:

- momentum continuation
- faster path dependency

- better environment for convexity if other layers align

8.5 6.5 Repository implementation

Gamma ideas flow through:

- `analytics/gamma_exposure.py`
- `analytics/gamma_flip.py`
- `analytics/market_gamma_map.py`
- `engine/trading_engine_support.py`

The engine uses these signals to form:

- `gamma_regime`
- `spot_vs_flip`
- `gamma_flip_distance_pct`
- downstream large-move and convexity overlays

9 7. Dealer Hedging Flows

9.1 7.1 Delta hedging mechanics

Suppose a dealer is short calls to customers. As spot rises, call delta rises. The dealer becomes synthetically short delta and must buy futures or underlying to hedge. This hedge buying can push spot even higher.

Conversely, if spot falls, delta falls and hedges are unwound.

In a short-gamma environment this creates positive feedback.

One may write the hedge adjustment very roughly as:

$$dH \approx -\Gamma dS$$

for a delta-hedged option book, where H denotes the hedge in the underlying. The sign and size of Γ determine whether hedging absorbs or amplifies the move.

This equation is intentionally schematic, but it captures the key point: hedge demand is endogenous to price movement.

9.2 7.2 Feedback loops

Hedging flow can create:

- upside acceleration
- downside acceleration
- pinning
- two-sided instability near a structural transition

These are not discretionary narratives. They are consequences of dynamic hedge rebalancing under convex exposure.

9.3 7.3 Pinning behavior

When positioning is concentrated near a strike and dealers are long gamma, hedging may stabilize price near that strike. This is called pinning.

Pinning matters because option buyers often need realized movement. A pinned market can be directionally "correct" in a broad sense while still being poor for option premium decay-adjusted returns.

9.3.1 EXAMPLE

Suppose spot oscillates repeatedly around a major ATM weekly strike with large open interest. A chart may show repeated tests of the level, but if dealer inventory and hedging keep pulling price back toward the strike, option buyers can lose through:

- time decay
- failed directional follow-through
- implied volatility compression after failed breakout attempts

9.4 7.4 Repository implementation

At the simplest level, `analytics/dealer_hedging_flow.py` estimates a buy/sell hedging direction from delta-weighted open interest. More advanced inference lives across:

- `analytics/dealer_hedging_flow.py`
- `analytics/dealer_hedging_simulator.py`
- `analytics/dealer_inventory.py`
- `risk/dealer_hedging_pressure_features.py`
- `risk/dealer_hedging_pressure_regime.py`

The dedicated dealer pressure overlay converts microstructure inputs into:

- `dealer_hedging_pressure_score`
- `dealer_flow_state`
- `upside_hedging_pressure`
- `downside_hedging_pressure`
- `pinning_pressure_score`

10 8. Volatility Dynamics

10.1 8.1 Realized volatility versus implied volatility

Realized volatility is the volatility observed in actual price changes. Implied volatility is the volatility level that rationalizes observed option prices through a pricing model.

The difference matters because:

- implied volatility reflects forward-looking demand and risk premia
- realized volatility measures what actually happened
- the relationship between the two drives option richness or cheapness

One common way to express realized volatility from log returns is:

$$ \hat{\sigma}_{realized} = \sqrt{\frac{252}{n-1}\sum_{t=1}^{n}(r_t-\bar{r})^2} $$

where r_t are daily log returns and the factor 252 annualizes the estimate.

The repository's live regime logic uses simpler practical proxies in some modules because the objective is robust state detection rather than textbook estimation elegance.

10.2 8.2 Volatility surface

Real options markets exhibit surfaces rather than a single volatility number. Implied volatility depends on:

- strike
- expiry
- underlying state

The repository contains supporting modules such as:

- `analytics/volatility_surface.py`
- `data/historical_iv_surface.py`

but the live engine often uses ATM IV and related proxies as tractable state variables rather than a fully dynamic surface model.

10.3 8.3 Compression versus expansion

Volatility is regime-dependent. Low realized volatility does not imply safety. It may imply latent instability if:

- dealers are short gamma
- the market is near the gamma flip
- liquidity is thin
- macro risk is rising

This is why the engine tracks both:

- volatility shock
- volatility compression
- volatility explosion probability

10.4 8.4 Expected move

A standard expected-move proxy is:

$$ \text{Expected Move} \approx S \cdot \sigma_{ATM} \cdot \sqrt{T} $$

where σ_{ATM} is ATM implied volatility and T is time to expiry in years.

This formula is used conceptually and operationally in the option efficiency layer.

10.4.1 INTUITION BEHIND THE FORMULA

If annualized volatility is σ, then over a horizon T years the standard deviation of returns in a diffusion approximation scales like:

$$ \sigma \sqrt{T} $$

Multiplying by spot converts return-volatility into point-volatility:

$$ \text{Move in points} \approx S \cdot \sigma \sqrt{T} $$

This expected move is not a forecast of the exact realized move. It is a scale parameter. It helps answer whether a target is plausible and whether an option premium is economically sensible.

10.4.2 EXAMPLE

If:

- $S = 22,000$
- $\sigma_{ATM} = 16\% = 0.16$
- $T = 7/365$

then:

$$ \text{Expected Move} \approx 22{,}000 \cdot 0.16 \cdot \sqrt{7/365} $$

which is roughly 487 points.

That does not mean the market "should" move 487 points. It means a one-standard-deviation-style horizon scale is on that order. A target requiring 900 points over the same horizon is immediately suspect unless other convexity or event arguments strongly support it.

10.5 8.5 Why volatility matters for option buying

Option buying benefits when:

- realized movement is large enough
- volatility expands or at least does not collapse
- the selected strike converts underlying movement into sufficient option gain

The repository's option efficiency overlay tries to answer exactly this question.

The implementation choice here is important. The repository does not attempt a full closed-form fair-value engine for every trade. Instead it uses expected move, delta mapping, premium coverage, and strike geometry as practical decision-support metrics. This is more robust than pretending one can estimate true theoretical mispricing from limited live inputs.

11 9. Liquidity Structure in Options

11.1 9.1 Gamma walls and liquidity walls

High open-interest strikes often function as structural walls. They are not guaranteed support or resistance, but they frequently represent concentrations of inventory and hedging importance.

The repository detects these in `analytics/gamma_walls.py` and related modules.

11.2 9.2 Liquidity voids and vacuums

A liquidity vacuum is a zone with sparse structural interest between better defended strikes. If price enters such a region, movement can become faster because there is less local resistance from inventory or liquidity concentration.

This is implemented in:

- `analytics/liquidity_vacuum.py`
- `analytics/liquidity_void.py`

The simplified intuition is:

- dense strikes can pin
- sparse strikes can accelerate

11.3 9.3 Clusters and path dependency

The path of price through strike space matters. A market leaving a high-OI cluster and moving into a sparse region may exhibit much larger realized movement than a chart-only model would suggest.

This is why liquidity structure is a separate state variable rather than a secondary annotation.

12 10. Macro and News Analysis

12.1 10.1 Why macro matters for options

Macroeconomic events can reprice both the expected direction and the expected distribution of outcomes. Options are therefore especially sensitive to:

- central bank decisions
- inflation releases
- employment releases
- geopolitical events
- earnings or policy shocks

12.2 10.2 Scheduled event risk

Scheduled event risk is conceptually simple but operationally important. Before a major event:

- implied volatility may rise
- liquidity may thin
- false breakouts may increase
- overnight holds may become less attractive

The repository implements scheduled-event logic in `macro/scheduled_event_risk.py` using configurable warning, lockdown, and cooldown windows.

12.3 10.3 Headline classification

The news subsystem uses deterministic keyword and category-based classification, not a black-box language model. This is deliberate: the goal is interpretability and predictable governance.

Headline classification in `news/classifier.py` maps headlines into:

- macro sentiment
- volatility shock bias
- India-specific macro bias
- global risk bias
- impact score

with category definitions in `news/keyword_rules.py` and category-level tuning overrides in `config/news_category_policy.py`.

This is an explicit implementation trade-off:

- deterministic rules are less expressive than large language models
- but they are easier to audit, backtest, tune, and govern

For a production research system, this is often the better first design.

12.4 10.4 Event risk logic

The macro/news stack does not act as an independent signal engine. It acts as:

- score adjustment
- confirmation modifier
- size cap input
- veto input under extreme event conditions

That design choice is critical. Macro is treated as a conditioning variable, not a replacement for microstructure logic.

13 11. Global Risk Factors

13.1 11.1 Why global markets matter for local index options

Indian index options do not trade in isolation. Overnight and cross-asset risk can enter through:

- oil prices
- US equities
- US yields
- USDINR
- VIX-like global volatility proxies
- precious and industrial commodities

These variables shape both risk sentiment and gap risk.

13.2 11.2 Cross-asset shock logic

The global risk feature model uses interpretable shock transformations, for example:

- oil shock score from 24-hour oil move
- gold risk score
- copper growth signal
- rates shock score
- currency shock score
- volatility compression and explosion features

These are built in `risk/global_risk_features.py` and transformed into regimes in `risk/global_risk_regime.py`.

The mathematical design here is intentionally piecewise rather than opaque. Shock variables are transformed through bounded functions and threshold maps so that researchers can answer:

- what caused the regime label?
- which cross-asset component dominated?
- was market data missing or stale?

This matters because macro/global overlays are among the easiest places to overfit if one hides complexity inside inscrutable models.

13.3 11.3 Global risk regimes

The system classifies global conditions into states such as:

- `GLOBAL_NEUTRAL`
- `RISK_OFF`
- `RISK_ON`
- `VOL_SHOCK`
- `EVENT_LOCKDOWN`

These are not directional trade signals. They are regime annotations used to:

- adjust confidence
- restrict overnight holding
- impose size caps
- veto trades under extreme conditions

14 12. Volatility Explosion and Convexity

14.1 12.1 Why volatility explosions occur

Volatility explosions are typically not random. They often arise when several conditions align:

- dealers are short gamma
- spot is near the gamma flip
- realized volatility had been compressed
- liquidity structure is thin
- macro or global risk increases sensitivity

This combination can create a convexity cascade.

14.2 12.2 Squeeze dynamics and air pockets

An upside squeeze occurs when:

- price rises
- dealer hedging requires more buying
- sparse resistance above spot permits acceleration

A downside air pocket occurs when:

- price falls through poorly defended structure

- dealer hedging requires more selling
- local liquidity is insufficient to slow the move

14.3 12.3 Gamma-vol feedback

The key intuition is:

1. spot starts moving
2. delta changes because of gamma
3. hedgers trade into that move
4. realized volatility rises
5. option markets reprice volatility
6. further hedging and positioning changes occur

This is why the repository includes a dedicated `risk/gamma_vol_acceleration_*` overlay rather than trying to force all such behavior into a single directional score.

One can write the intuition schematically as:

$$\text{Acceleration Risk} \sim f(\text{short gamma}, \text{flip proximity}, \text{vol compression}, \text{vacuum}, \text{macro stress})$$

The repository turns this qualitative relationship into deterministic bounded feature scores rather than a hidden nonlinear black box. That is a practical institutional choice: interpretable convexity logic is often more actionable than an opaque classifier.

15 13. Signal Generation Framework

15.1 13.1 Philosophy

The engine uses a layered signal framework. It does not ask a single model to map raw data directly to a trade. Instead it:

1. constructs microstructure state
2. constructs probability and directional evidence
3. scores trade strength
4. applies confirmation filters
5. applies overlays and risk layers
6. selects strikes and trade geometry

15.2 13.2 Flow signals

The engine uses options-flow imbalance and smart-money-style flow inference from the option chain and related analytics:

- `analytics/options_flow_imbalance.py`
- `analytics/smart_money_flow.py`

These features help answer whether activity is aligned with a bullish or bearish move and whether the flow is institutionally meaningful.

15.3 13.3 Gamma regime signals

Gamma regime enters in multiple places:

- directional evidence
- large-move probability
- confirmation context
- convexity overlays

This is consistent with the system's thesis that gamma is structural, not decorative.

15.4 13.4 Volatility regime signals

The live engine uses volatility state as both a risk and opportunity variable. High-vol expansion may support option buying if move magnitude and path support also align. Low-vol compression may either suppress movement or precede a breakout.

15.5 13.5 Liquidity signals

Liquidity signals include:

- vacuum states
- void structure
- wall proximity
- gamma cluster interactions

These features are used both in trade strength and in strike selection.

15.6 13.6 Trade strength scoring

`strategy/trade_strength.py` builds an additive score from:

- flow alignment
- smart-money flow
- gamma regime
- spot relative to gamma flip
- hedging bias
- wall proximity
- liquidity map features
- move-model output
- directional consensus

This is not a probabilistic theorem; it is an interpretable evidence aggregation framework.

One can think of the score abstractly as:

$$\text{Trade Strength} = \sum_j w_j x_j + \sum_k b_k \mathbf{1}_{\text{alignment}_k}$$

where:

- x_j are normalized evidence components
- w_j are governed weights
- b_k are alignment bonuses or conflict penalties

The key implementation choice is that these weights are now registry-governed rather than buried in scattered constants. This matters for auditability and tuning discipline.

15.7 13.7 Confirmation filters

`strategy/confirmation_filters.py` acts as a secondary layer:

- open versus spot alignment
- previous close alignment
- intraday range expansion
- flow confirmation
- hedging confirmation
- gamma event confirmation
- move-probability confirmation
- flip alignment

The key philosophical point is that confirmation does not create direction. It only strengthens, weakens, or vetoes a direction proposed by the primary engine.

This is a strong design choice. Many trading systems collapse signal and confirmation into one score and then lose the ability to explain why a trade was vetoed. Here, confirmation remains conceptually subordinate, which keeps the logic easier to inspect.

16 14. Trade Construction

16.1 14.1 From signal to trade

A signal becomes a trade through several decisions:

- choose direction
- choose call or put
- choose strike
- estimate entry, target, stop
- apply budget constraints
- decide whether it is tradeable, watchlist-only, or blocked

16.2 14.2 Direction determination

Direction is determined in the core engine path through a vote-like combination of evidence from flow, gamma context, hedging bias, and move probability. Implementation lives mainly in:

- `engine/trading_engine_support.py`
- `strategy/trade_strength.py`

16.3 14.3 Strike selection

`strategy/strike_selector.py` ranks candidate strikes using:

- moneyness preferences
- directional side preference
- premium band preferences
- liquidity
- wall distance
- gamma cluster distance
- IV preferences

- optional candidate scoring hooks from overlays

This is important because a correct directional view can still fail as an options trade if strike selection is poor.

16.3.1 EXAMPLE

Suppose the engine sees a bullish move likely to develop over the next 60 minutes.

- A far OTM call may have cheap premium but require too much intrinsic distance to be reached.
- A deep ITM call may respond well directionally but be too capital intensive.
- A near-ATM call may offer the best balance of delta, gamma, and premium efficiency.

The repository's strike selector operationalizes exactly this kind of trade-off using moneyness, liquidity, wall distance, IV, and candidate-score hooks.

16.4 14.4 Capital constraints

`strategy/budget_optimizer.py` adjusts lot sizing against available capital. This preserves the separation between signal quality and practical trade feasibility.

16.5 14.5 Target and stop logic

The baseline target and stop model in `strategy/exit_model.py` is simple:

$$ \text{Target} = P_0 \left(1 + \frac{\tau}{100}\right) $$

$$ \text{Stop} = P_0 \left(1 - \frac{s}{100}\right) $$

where τ is target profit percent and s is stop loss percent.

This is intentionally simple and should be interpreted as research geometry rather than a full execution optimizer.

17 15. Research and Backtest Framework

17.1 15.1 Research as signal evaluation

The repository's research philosophy is not "did the executed trade make money?" It is "was the signal useful relative to what the market actually did after the signal?"

This is implemented through the canonical signal evaluation dataset in:

- `research/signal_evaluation/dataset.py`

- `research/signal_evaluation/evaluator.py`

Each signal is assigned a stable `signal_id`, captured once, and enriched over time with subsequent outcomes.

17.2 15.2 Outcome horizons

The evaluator tracks horizons such as:

- 5 minutes
- 15 minutes
- 30 minutes
- 60 minutes
- session close
- next open
- next close

This allows the same signal to be judged under multiple holding frameworks.

17.3 15.3 Composite scoring

The research framework builds evaluation metrics such as:

- direction score
- magnitude score
- timing score
- tradeability score
- composite signal score

This is important because no single outcome measure captures all useful dimensions of a signal.

In abstract form:

$$\text{Composite Signal Score} = w_d \cdot \text{Direction Score} + w_m \cdot \text{Magnitude Score} + w_t \cdot \text{Timing Score} + w_q \cdot \text{Tradeability Score}$$

with weights governed through `config/signal_evaluation_scoring.py`.

This is a research design choice, not a universal truth. The advantage is that it produces a multidimensional evaluation surface rather than over-optimizing for one narrow target such as raw short-horizon hit rate.

17.4 15.4 Backtesting

The repository includes backtest tooling in `backtest/`, including:

- replay
- parameter sweeps
- Monte Carlo support
- PnL engines
- scenario runners

However, the research center of gravity is still the canonical signal dataset.

17.5 15.5 Tuning methodology

The tuning stack includes:

- parameter registry
- parameter packs
- experiment runner
- search strategies
- walk-forward validation
- regime-aware validation
- promotion and shadow mode

This is a significant design choice. It means parameter research is governed and auditable rather than embedded ad hoc in scattered constants.

The objective framework in `tuning/objectives.py` explicitly combines:

- hit rate
- composite signal quality
- tradeability
- target reachability
- signal frequency
- drawdown proxy
- penalties for instability and over-selectivity

This is conceptually close to a constrained research optimization problem:

$$\max_{\theta \in \Theta} \mathcal{J}(\theta)$$

subject to:

$$\text{frequency}(\theta) \geq f_{min}, \quad \text{robustness}(\theta) \geq r_{min}, \quad \text{sample count}(\theta) \geq n_{min}$$

where θ denotes a parameter pack.

That framing is important because it resists the common failure mode of tuning only for PnL or only for one validation window.

17.6 15.6 Overfitting danger

Overfitting is a central risk because:

- options data is regime-sensitive
- convexity regimes are sparse
- event-driven outcomes cluster
- selection thresholds can bias which signals are evaluated

The repository mitigates this through:

- walk-forward validation
- regime-aware validation
- robustness metrics
- promotion criteria
- shadow mode

18 16. System Architecture

18.1 16.1 High-level architecture

The repository can be thought of as the following pipeline:

```
data ingestion -> normalization -> analytics -> core engine ->
overlay/risk layers -> strike/trade construction -> signal capture ->
evaluation -> tuning/validation/promotion
```

An equally useful way to visualize the repository is as two coupled systems:

```
System A: live/replay inference
data -> app runtime context -> analytics -> engine -> overlays ->
execution_trade + trade_audit + legacy trade payload

System B: research governance
trade payload -> canonical dataset -> evaluation -> tuning -> validation ->
promotion / shadow review
```

This decomposition is one of the strongest architectural properties of the repository. It keeps market-state generation separate from parameter governance.

18.2 16.2 Analytics module

`analytics/` contains market-state inference modules such as:

- Greeks engine
- gamma exposure
- gamma flip
- gamma walls
- dealer hedging flow
- liquidity vacuum / void
- volatility regime
- volatility surface support

This package translates raw option-chain information into interpretable state variables.

18.3 16.3 Engine module

`engine/` contains orchestration and runtime shaping:

- `engine/signal_engine.py`
- `engine/trading_engine.py`
- `engine/trading_engine_support.py`
- `engine/runtime_metadata.py`

This is the layer that turns analytics state into a coherent trade payload, now with explicit execution-facing and audit-facing subviews.

18.4 16.4 Strategy module

`strategy/` contains:

- trade strength logic
- confirmation filters
- strike selection
- budget optimization
- exit geometry

This is the bridge between state inference and trade construction.

18.5 16.5 Data module

`data/` manages:

- provider adapters
- normalization
- replay loading
- shared preloaded-snapshot orchestration in `app/engine_runner.run_preloaded_engine_snapshot(...)`
- expiry resolution
- historical option-chain loading
- global market snapshots

This layer creates a consistent data interface for the engine.

18.6 16.6 Models module

`models/` contains:

- feature builders
- rule-based move probability
- ML move predictor support

This is where predictive submodels and feature transformations live.

18.7 16.7 Backtest module

`backtest/` contains:

- historical replay

- intraday backtest logic that now reuses the shared preloaded runner path
- parameter sweep utilities
- PnL logic
- scenario runners

This is the historical simulation and scenario-validation layer.

18.8 16.8 Visualization module

The user requested a visualization module in the conceptual architecture. In the current repository, there is no standalone `visualization/` package remaining. Visualization responsibilities are effectively split across:

- `app/streamlit_app.py`
- terminal output in `main.py`
- runtime metadata formatting in `engine/runtime_metadata.py`
- rendered documentation under `documentation/`

So conceptually a visualization layer exists, but in the current repository it is embedded in the application and documentation surfaces rather than isolated as a separate package.

18.9 16.9 Risk and overlay modules

`risk/` contains dedicated overlay layers:

- global risk
- gamma-vol acceleration
- dealer hedging pressure
- option efficiency

These do not replace the core signal engine. They modify and qualify it.

18.10 16.10 Tuning and governance modules

`tuning/` contains:

- registry and packs
- objectives and experiments
- search
- campaigns
- walk-forward validation

- regime labeling
- promotion
- shadow mode
- reporting

This makes the system unusual among retail-style options engines. It is designed as a research-governance system as much as a signal engine.

19 17. Implementation Details

This section ties major concepts to concrete repository seams.

19.1 17.1 Options pricing and Greeks

19.1.1 THEORY

Options are nonlinear derivatives priced under arbitrage logic and characterized by sensitivities such as delta, gamma, theta, vega, vanna, and charm.

19.1.2 TRADING INTUITION

Dealers hedge these exposures, and those hedges move markets.

19.1.3 REPOSITORY IMPLEMENTATION

- `analytics/greeks_engine.py`
- `analytics/gamma_exposure.py`
- `analytics/gamma_flip.py`

These modules compute both contract-level sensitivities and structural aggregates.

19.1.4 WHY THIS IMPLEMENTATION IS REASONABLE

The repository chooses explicit formula-based Greeks rather than hidden vendor Greeks because:

- the formulas are auditable
- the assumptions are known
- the outputs can be recomputed consistently across data providers
- research and tuning can be done on a stable internal representation

19.2 17.2 Microstructure state collection

19.2.1 THEORY

The option chain contains inventory structure and directional/convexity information.

19.2.2 TRADING INTUITION

Strikes with concentrated open interest or sparse neighboring liquidity can change path behavior materially.

19.2.3 REPOSITORY IMPLEMENTATION

`engine/trading_engine_support.py` collects market state by invoking analytics modules and normalizing the chain first. This is where the engine assembles:

- gamma regime
- spot versus flip
- hedging bias
- flow state
- wall state
- vacuum state
- volatility regime

19.2.4 WHY THE ORCHESTRATION IS CENTRALIZED

`engine/trading_engine_support.py` serves as a state assembly layer so that the public engine entrypoint does not directly encode all analytics details. This is a strong software-engineering choice because it:

- keeps the main orchestration readable
- makes analytics upgrades more localized
- preserves backward compatibility for callers of `generate_trade()` while exposing `execution_trade` and `trade_audit` in runtime payloads

19.3 17.3 Trade strength

19.3.1 THEORY

A robust signal often comes from multiple moderately informative pieces of evidence rather than one dominant factor.

19.3.2 TRADING INTUITION

If flow, hedging bias, gamma regime, and spot-vs-flip all align, the signal is stronger than if only one of them does.

19.3.3 REPOSITORY IMPLEMENTATION

`strategy/trade_strength.py` scores these evidence blocks explicitly and returns a score plus a breakdown.

19.3.4 EXAMPLE OF IMPLEMENTATION PHILOSOPHY

Instead of hiding score formation in one model probability, the repository keeps named components such as:

- `flow_signal_score`
- `hedging_bias_score`
- `gamma_regime_score`
- `move_model_score`
- `directional_consensus_score`

This makes post-trade research and debugging much easier.

19.4 17.4 Confirmation layer

19.4.1 THEORY

A primary model and a secondary confirmation model reduce noise better than a single monolithic score.

19.4.2 TRADING INTUITION

A call signal that is also above the open, above previous close, supported by flow, and consistent with spot-vs-flip is more convincing.

19.4.3 REPOSITORY IMPLEMENTATION

`strategy/confirmation_filters.py`

19.4.4 WHY THIS IS NOT MERGED INTO TRADE STRENGTH

If confirmation were absorbed entirely into the base score, it would become harder to distinguish:

- strong signal with weak confirmation
- weak signal with good confirmation

- strong signal vetoed by a conflict cluster

The current design preserves these distinctions.

19.5 17.5 Global risk layer

19.5.1 THEORY

Local options do not trade independently of cross-asset risk.

19.5.2 TRADING INTUITION

Overnight holds or directional confidence should be reduced if VIX, oil, US equities, yields, and event risk are collectively unstable.

19.5.3 REPOSITORY IMPLEMENTATION

- `risk/global_risk_features.py`
- `risk/global_risk_regime.py`
- `risk/global_risk_layer.py`

The layer produces regime labels, risk scores, overnight permissions, and engine modifiers.

19.5.4 IMPLEMENTATION RATIONALE

The layer is a facade over:

- raw feature extraction
- regime classification
- policy evaluation

This separation allows researchers to inspect:

- raw features
- derived state
- final decision

without conflating them.

19.6 17.6 Gamma-vol acceleration layer

19.6.1 THEORY

Moves accelerate when short gamma, flip proximity, volatility transition, and thin liquidity interact.

19.6.2 TRADING INTUITION

This layer identifies whether a "normal" directional move might become a squeeze, air pocket, or volatility event.

19.6.3 REPOSITORY IMPLEMENTATION

- `risk/gamma_vol_acceleration_features.py`
- `risk/gamma_vol_acceleration_regime.py`
- `risk/gamma_vol_acceleration_layer.py`

19.7 17.7 Dealer hedging pressure layer

19.7.1 THEORY

Dealer hedging can reinforce, suppress, or pin spot depending on inventory and location relative to structural levels.

19.7.2 TRADING INTUITION

A trade aligned with strong upside hedging pressure is more attractive than one fighting pinning.

19.7.3 REPOSITORY IMPLEMENTATION

- `risk/dealer_hedging_pressure_features.py`
- `risk/dealer_hedging_pressure_regime.py`
- `risk/dealer_hedging_pressure_layer.py`

19.8 17.8 Option efficiency layer

19.8.1 THEORY

A good directional signal can still be a poor option trade if expected move, premium, and strike geometry are unfavorable.

19.8.2 TRADING INTUITION

You want to know whether the market is likely to move enough, fast enough, to justify buying the chosen option.

19.8.3 REPOSITORY IMPLEMENTATION

- `risk/option_efficiency_features.py`

- `risk/option_efficiency_layer.py`

This computes expected move, target reachability, premium efficiency, strike efficiency, and overnight option-efficiency penalties.

19.8.4 IMPLEMENTATION RATIONALE

The repository intentionally avoids claiming a full theoretical option-pricing mispricing signal here. Instead it uses practical overlays such as:

- expected move coverage
- premium coverage
- strike-distance efficiency
- payoff geometry hints

This is a stronger design than a fake precision model built on weak inputs.

19.9 17.9 Research dataset

19.9.1 THEORY

A systematic engine should be judged on what it predicted and what the market did afterward, not on a small and noisy sample of executed trades.

19.9.2 TRADING INTUITION

Signal evaluation is a cleaner research target than fill-dependent PnL.

19.9.3 REPOSITORY IMPLEMENTATION

- `research/signal_evaluation/dataset.py`
- `research/signal_evaluation/evaluator.py`

The dataset is canonical, deduplicated, and enriched over time.

19.10 17.10 Parameter governance

19.10.1 THEORY

A multi-layer engine accumulates many thresholds, weights, and penalties. Without governance, tuning becomes opaque and fragile.

19.10.2 TRADING INTUITION

A desk-quality system needs controlled calibration, not hand-edited constants scattered across files.

19.10.3 REPOSITORY IMPLEMENTATION

- `tuning/registry.py`
- `tuning/packs.py`
- `config/policy_resolver.py`
- `tuning/runtime.py`
- `tuning/objectives.py`
- `tuning/experiments.py`
- `tuning/validation.py`
- `tuning/promotion.py`
- `tuning/shadow.py`

19.10.4 WHY THIS MATTERS TECHNICALLY

In many trading repositories, parameters are dispersed across constants, spreadsheets, notebooks, and ad hoc scripts. Here, the parameter-governance stack turns calibration into a first-class subsystem. That makes the engine easier to evolve without losing research traceability.

20 18. Limitations

20.1 18.1 Model assumptions

The system depends on approximations:

- Black-Scholes-style Greeks are only local approximations
- aggregate dealer positioning is inferred, not observed directly
- option-chain-based realized volatility proxies are imperfect
- expected move is a practical approximation, not a full surface-consistent pricing forecast

There is also an important philosophical limitation: the engine infers structural pressure from public market observables. It does not observe the internal books of large dealers, proprietary flow-routing information, or true institutional hedge mandates.

20.2 18.2 Data limitations

The quality of inference depends on:

- provider data cleanliness
- chain completeness
- IV quality
- correct call-put pairing
- expiry metadata
- historical availability for replay and research

For some providers, chain quality may differ materially by:

- intraday timestamp
- expiry bucket
- option depth from ATM
- availability of greeks or IV fields

This creates heteroskedastic data quality, which is difficult to model perfectly.

20.3 18.3 Parameter sensitivity

The engine contains many interacting layers. Even with the registry and validation framework, parameter sensitivity remains real, particularly in:

- global risk blends
- convexity overlays
- dealer pressure state thresholds
- strike-selection heuristics

The existence of a large governed tuning surface is a strength, but it also creates a combinatorial search problem. Many parameter groups may be weakly identified in low-sample regimes, especially:

- volatility shock states
- overnight convexity events
- rare macro-event dislocations

20.4 18.4 Execution limitations

The repository currently does not include a live order-routing engine. This is a design boundary, not an oversight, but it means:

- slippage is not modeled through actual fills
- live execution frictions are external
- order management and trade lifecycle automation are outside current scope

This means research conclusions should be interpreted carefully. A signal can be genuinely good in a market-forecasting sense while still requiring a separate execution layer to realize value in practice.

20.5 18.5 Regime dependency

Any options engine is regime dependent. Relationships that hold in:

- short gamma weeks
- event-heavy periods
- panic-driven volatility shocks

may weaken in:

- long gamma pinning regimes
- quiet carry environments
- structurally suppressed realized volatility periods

This regime dependency also creates a model-governance problem: a parameter pack that is strong in one regime may be dangerously fragile in another. This is why the repository's walk-forward and regime-aware validation framework is not a luxury feature but a necessity.

20.6 18.6 Structural simplifications

Some modules intentionally favor robustness and interpretability over maximal theoretical completeness. This is a strength operationally, but it means the engine is not:

- a complete stochastic-volatility pricer
- a full dealer inventory reconstruction model
- an optimal execution engine
- a fully market-impact-aware simulator

20.7 18.7 Statistical and research limitations

Even with the signal-evaluation-first architecture, the research program still faces classical quantitative limitations:

- nonstationarity of market structure
- regime imbalance in the historical sample
- sparse tails for crisis-like observations
- potential circularity if evaluation-selection thresholds are over-tuned
- limited inferential power for high-dimensional parameter interactions

These limitations are especially important for advanced tuning. A rich search space does not guarantee that the data can identify the correct parameter set.

20.8 18.8 Software and operational limitations

The codebase is architecturally disciplined, but operational complexity still exists:

- multiple providers can produce slightly different chain shapes
- scenario fixtures may lag production logic if not maintained carefully
- documentation and rendered artifacts can become stale if not regenerated
- local environment differences can affect replay and rendering tools

These are not exotic theoretical problems, but they matter for a production research workflow.

21 19. Future Enhancements

21.1 19.1 Improved volatility modeling

Future work could include:

- richer volatility surface estimation
- skew-aware expected-move estimation
- local-vol or stochastic-volatility inspired state variables
- realized/implied spread diagnostics

More specifically, promising research directions include:

- SABR-style or local-vol-inspired surface summaries
- event-specific IV term-structure dislocations
- cross-sectional skew curvature diagnostics

- realized-vol forecasting conditioned on gamma and liquidity state

21.2 19.2 Better dealer positioning estimates

Potential future improvements:

- more sophisticated sign conventions for GEX
- improved inventory inference
- expiry-specific dealer maps
- dynamic path simulations under alternative spot trajectories

Longer term, one could also study:

- sensitivity of inferred dealer pressure to assumptions about customer positioning
- separate weekly versus monthly expiry dealer maps
- pathwise hedge-demand simulation under alternative spot shocks
- stress testing of flip-level uncertainty

21.3 19.3 Machine learning extensions

The repository is already structurally ready for:

- surrogate optimization of parameter packs
- regime-conditional tuning
- ML pack recommendation in shadow mode
- eventually neural meta-models

but only within the existing signal-evaluation-first and governance framework.

The most credible path is likely:

1. surrogate optimization over parameter packs
2. regime-conditioned pack recommendation
3. neural meta-models only after the experiment ledger is sufficiently large

This ordering matters. It respects the asymmetry between explainability and sample complexity.

21.4 19.4 Improved news analysis

Potential upgrades:

- better entity extraction
- temporal clustering of headline shocks

- more nuanced macro-event surprise modeling
- multilingual or richer structured news feeds

One especially useful direction would be explicit surprise modeling for scheduled macro events, where the magnitude and direction of surprise are measured relative to consensus, rather than treating all high-severity events as structurally similar.

21.5 19.5 Improved option efficiency models

Potential directions:

- better strike-by-strike payoff efficiency measures
- richer delta and convexity mapping
- volatility-surface-aware strike selection
- regime-conditioned option richness filters

Further work could also add:

- better premium-to-expected-payoff mapping by moneyness bucket
- expiry-aware option-buying efficiency curves
- realized slippage-aware strike quality metrics
- theta-adjusted overnight carry efficiency measures

21.6 19.6 Capital allocation systems

The current system is primarily single-trade signal centric. A future portfolio layer could address:

- cross-signal capital allocation
- correlation-aware sizing
- convexity budget management
- drawdown-aware exposure throttling

21.7 19.7 Portfolio-level and cross-signal research

At present the repository is strongest at single-signal evaluation. A natural future extension is a portfolio layer that studies:

- interaction among simultaneous signals
- overlap of macro risk and convexity exposures
- correlation of edge across symbols and expiries
- optimal diversification of limited convexity budget

21.8 19.8 Execution and transaction-cost research

Once the signal layer is mature, future work could include:

- paper execution simulation
- queue-aware fill assumptions
- spread dynamics by strike and time of day
- execution-quality attribution versus signal-quality attribution

21.9 19.9 Research infrastructure enhancements

The repository already contains a strong tuning and governance framework, but further infrastructure work could add:

- richer experiment meta-datasets
- surrogate-model-assisted tuning campaigns
- automated research memo generation
- calibration drift monitoring
- stronger model-risk dashboards

22 20. Assumptions Used in Interpreting the Repository

The following assumptions were used when writing this document:

1. The current repository boundary is signal generation, research evaluation, and governance; it is not a live automated execution platform.
2. The canonical truth source for research is the signal evaluation dataset in `research/signal_evaluation/`, not discretionary or manually executed trades.
3. Dealer positioning is estimated from option-chain and Greek proxies rather than from proprietary dealer inventory data.
4. The current historical/backtest environment is useful for research, and orchestration parity is materially improved through `run_preloaded_engine_snapshot(...)`, but the signal evaluation dataset remains the primary validation substrate.
5. The user-requested "visualization module" is conceptually present but is currently implemented through application and documentation layers rather than a dedicated `visualization/` package.

23 Concluding Remark

`options_quant_engine` should be understood as a layered quantitative options research system. Its intellectual center is not a single prediction function, but a structured attempt to model:

- option convexity
- dealer hedging behavior
- liquidity structure
- volatility state
- macro and global risk
- trade efficiency

within a disciplined signal-evaluation and governance loop.

That combination is what makes the system closer to an institutional microstructure-and-convexity research platform than to a conventional indicator-driven trading script.